Operations Manual

OTS Manager CLI & Batch

Management of identities, groups, and QR Codes via REST API on OpenTAKServer

---

Document Control

Field	Information
Document	Operations Manual — OTS Manager CLI & Batch
Version	**1.0.1**
Author	**Orlando Nascimento Santos**
Email	**onascimento@gmail.com**
Platform	OpenTAKServer (OTS)
Interface	Python, CLI, and REST API
Language	English

> **Objective.** This manual presents the installation, configuration, operation, batch provisioning, QR Code generation, diagnostics, and best practices for the `ots_manager.py` utility.

Table of Contents

1. Overview
2. Architecture and operational flow
3. Prerequisites and installation
4. Configuration via environment variables
5. Security and credential protection
6. REST API mapping
7. CLI commands
8. Batch provisioning
9. QR Codes and access validity
10. Operational procedures
11. Error handling and diagnostics
12. Best practices and limitations
13. Operations checklist
14. Quick reference
15. Appendix: code structure

What's New in Version 1.0.1

- **Per-group directions:** `IN`, `OUT`, or `BOTH`, using `GROUP:DIRECTION` in the CLI or JSON objects with `name` and `direction`.
- **Group listing:** `list-groups` queries and displays the groups existing on OpenTAKServer.
- **User listing:** `list-users` displays `username`, administrator status, and `last_login` when available.
- **Group expansion:** `ALL`, `ALL:IN`, and `ALL:OUT` associate a user with every group returned by the server.
- **Backward compatibility:** plain group strings remain valid and default to `BOTH`.
- **Optimized payload:** `exp` and `max` are sent only when provided.
- **Security compliance:** protected operations send `Referer`, `Origin`, and CSRF tokens.

1. Overview

OTS Manager is a Python utility for automating OpenTAKServer administration. It encapsulates the operations required to:

- authenticate to the backend via `/api/login`;
- maintain an HTTP session with cookies;
- retrieve and send CSRF tokens required by Flask-WTF/Flask-Security;
- send the `Referer` and `Origin` headers;
- create groups;
- create users with mandatory password confirmation;
- associate users with groups in the `IN` and `OUT` directions;
- generate QR Code strings and PNG images for Android and iPhone;
- process multiple users from a JSON file;
- consolidate the processing results into a JSON report.
- retrieve the list of existing groups from the server (`list_groups()`) and provide a `list-groups` CLI command;
- support the special group value `ALL`: when `ALL` is supplied as a group name (CLI or batch), the utility expands it to the full list of groups returned by the server and associates the user with each group.
- list users and show `admin` status and `last_login` via the `list-users` CLI command;

1.1 Operating model

The recommended workflow is:

1. configure `OTS_URL`, `OTS_USER` and `OTS_PASS`;
2. validate connectivity with the server;
3. perform the login;
4. create the required groups;
5. create the user;
6. associate the user with groups in `IN` and `OUT`;
7. generate the QR Code for the correct ecosystem;
8. store the PNG and record the result;
9. validate activation on the ATAK/iTAK device.

1.2 Ecossistemas suportados

Application	Endpoint	Operation	Result
Android / ATAK	`/api/atak_qr_string`	`POST`	QR Code configuration string
iPhone / iTAK	`/api/itak_qr_string`	`GET`	iOS configuration string

2. Architecture and operational flow

The script uses `requests.Session()` to preserve cookies and headers between calls. After login, the CSRF token can be obtained from the cookies `csrf_token`, `csrf_access_token`, or `XSRF-TOKEN`. If the server returns the token in the response body, the utility also attempts to retrieve it from the JSON.

2.1 Authentication flow

```
ots_manager.py
      |
      | POST /api/login
      v
OpenTAKServer
      |
      | session cookies + CSRF token
      v
requests.Session()
      |
      +--> Referer / Origin
      +--> X-CSRFToken / X-CSRF-TOKEN
      +--> protected operations
```

2.2 Applied headers

The expected global headers are:

```
Content-Type: application/json
Referer: <OTS_URL>/
Origin: <OTS_URL>
```

For protected operations, the token is additionally sent as:

```
X-CSRFToken: <token>
X-CSRF-TOKEN: <token>
```

3. Prerequisites and installation

3.1 Requirements

- Python 3.9 or higher;
- network access to OpenTAKServer;

- credentials for an account authorized to create users and groups;
- write permissions in the working directory;
- `pip` available in the Python environment.

3.2 Dependency installation

```
python3 -m venv .venv
. .venv/bin/activate
python -m pip install --upgrade pip
pip install requests qrcode pillow
```

The libraries have the following responsibilities:

Library	Function
`requests`	HTTP communication, session, and cookies
`qrcode`	QR Code image generation
`Pillow`	Image backend used by `qrcode`

3.3 Suggested organization

```
ots-manager/
├── ots_manager.py
├── usuarios.json
├── qrcodes/
├── resultado_qr_codes.json
└── .venv/
```

Do not version files containing passwords, tokens, QR strings, or real personal data.

4. Configuration via environment variables

Set the parameters before execution:

```
export OTS_URL="http://opentakserver.example.com"
export OTS_USER="admin"
export OTS_PASS="your_password_here"
```

> **Note — local execution without Nginx:** if OpenTAKServer is run locally, directly through the API and without Nginx, include the API port in `OTS_URL`. The default API port is **8081** (for example, `http://localhost:8081`). When accessing through Nginx, use the URL published by the proxy, normally without specifying a port.

The script removes the trailing slash from `OTS_URL`. If the variables are absent, the default values are:

Variable	Code default	Recommendation
`OTS_URL`	`http://localhost`	Always specify explicitly
`OTS_USER`	`admin`	Use a named or service account
`OTS_PASS`	`admin_password`	Never rely on the default

4.1 Connectivity test

```
curl -i "$OTS_URL/"
```

If the service requires HTTPS, use an `https://` and validate the certificate. Avoid disabling TLS validation in production.

4.2 Running without exposing the password in shell history

Whenever possible, provide the password through a secure environment mechanism. At a minimum, avoid placing the password directly in shared commands or versioned scripts:

```
read -r -s OTS_PASS
export OTS_PASS
python ots_manager.py --help
```

5. Security and credential protection

> **Warning:** the original example uses HTTP and an IP address. In production, prefer HTTPS with a valid certificate, a restricted network, and an account with the least privilege necessary.

Best practices:

- do not store `OTS_PASS` no repositório;
- do not share `resultado_qr_codes.json` without removing QR strings;
- protect generated PNG files because they may enable device provisioning;
- use service accounts with password rotation;
- restrict server access using a firewall or VPN;
- record operations without recording passwords or tokens;
- validate the destination before running destructive or provisioning commands;
- remove temporary files containing credentials after use.

6. REST API mapping

Operation	Endpoint	Method	Main data	CSRF/Referer
Authentication	`/api/login`	`POST`	`username`, `password`	Initial session
Create group	`/api/groups`	`POST`	`name`	Yes
Create user	`/api/user/add`	`POST`		

Operation	Endpoint	Method	Main data	CSRF/Referer
			`username`, `password`, `confirm_password`, `email`, `administrator`	According to OTS configuration
Link group	`/api/users/groups`	`PUT`	`username`, `groups[]`, `direction`	According to OTS configuration
QR Android	`/api/atak_qr_string`	`POST`	`username`, `exp`, `nbf`, `max`	Yes
QR iPhone	`/api/itak_qr_string`	`GET`	defined by the server	Authenticated session

6.1 Required and optional fields

The table below consolidates the fields accepted pelo `ots_manager.py`. Fields marked as **opcionais** may be omitted; in which case the script uses the CLI default or lets OpenTAKServer apply its own policy.

Operation	Required fields	Optional fields and default behavior
Authentication — `/api/login`	`username`, `password`	None. They are obtained from `OTS_USER` and `OTS_PASS`; o código possui valores padrão, mas recomenda-se configure ambos explicitamente.
Create group — `/api/groups`	`name`	Nenhum.
Create user — `/api/user/add`	`username`, `password`, `confirm_password`	`email` **is not required** and may be omitted or left empty; `administrator` is optional and defaults to `false`. O `confirm_password` é preenchido automaticamente pelo script com o mesmo valor de `password`.
Link group — `/api/users/groups`	`username`, `groups[]` with at least one group, `direction`	In the CLI, `direction` is optional and defaults to `BOTH`, performing one `IN` and onand `OUT` association.
QR Android — `/api/atak_qr_string`	`username`	`` `exp `` and `max` are **optional**. `nbf` is calculated automatically only when `exp` is provided. If `exp` and `max` are omitted or `null`, they are not sent and the server default policy applies.
QR iPhone — `/api/itak_qr_string`	Authenticated session	The `GET` request has no user-supplied fields; the configuration is returned by the server.
CLI `create-user`	`--username`, `--password`	`--email`, `--groups`, `--admin`, `--app`, `--exp`, `--max` e `--save-qr` are optional. `--app` defaults to `android`; `--admin` defaults to false.
CLI `qr`	`--username`	`--app`, `--exp`, `--max` e `--save-qr` are optional. `--app` defaults to `android`.
CLI `create-group`	`--name`	Nenhum.
CLI `link`	`--username`, `--group`	`--direction` is optional and defaults to `BOTH`.
CLI `batch`	`--file`	`--output` is optional and defaults to `resultado_qr_codes.json`. Dentro de cada registro, `username` e `password` are required; `email`, `administrator`, `groups`, `app`, `expiration` and `max_uses` are optional.

Batch user field summary

Field JSON	Required?	Notes
`username`	**Yes**	User identifier in OTS.
`password`	**Yes**	Initial password; the script also sends `confirm_password` com o mesmo valor.
`email`	**No**	May be omitted, set to `null` or an empty string, depending on the OTS installation validation. The script sends it as empty when not provided.
`administrator`	**No**	Assume `false` when omitted.
`groups`	**No**	Group list; when provided, each group is associated according to its direction (`IN`, `OUT`, or `BOTH`).
`app`	**No**	Assume `android`; also accepts `iphone`.
`expiration`	**No**	Days or date `YYYY-MM-DD`; when absent or `null`, no expiration is sent.
`max_uses`	**No**	Activation limit; when absent or `null`, no limit is sent.

> **Important:** `email`, `expiration`, and `max_uses` are not required fields. The absence of these fields does not prevent user creation or QR Code generation; in these cases, the server applies its default values and policies. However, `username` and `password` are essential for each user. The field `confirm_password` is required by the API, but is generated automatically by the script and does not need to be supplied separately in the CLI or JSON file.

6.2 Successful responses

Depending on the operation, the utility treats status codes `200`, `201` and `204`. An existing group or user may be treated as an idempotent condition when the response `400` contains a duplication indication, como `exists`.

6.2 Group directions

The association is performed separately:

- `IN`: messages received by the user;
- `OUT`: messages sent by the user;
- `BOTH`: CLI shortcut that executes two requests, one for each direction.

For bidirectional tactical traffic, always validate `IN` and `OUT`.

6.3 Group directions in the CLI

Each group can receive an individual direction using the `GROUP:DIRECTION` syntax:

- `IN`: the user receives messages from the group;
- `OUT`: the user sends messages to the group;
- `BOTH`: enables both directions.

When the suffix is omitted, the direction defaults to `BOTH`.

```
# Groups with different directions
python ots_manager.py create-user -u "pilot1" -p "Pass123!" \\
  -g CSAR:IN Rescue:OUT Command:BOTH --app android

# Without a suffix, the group automatically uses BOTH
python ots_manager.py create-user -u "pilot2" -p "Pass123!" \\
  -g CSAR Aviation:IN --app android

# Expiration in days and activation limit
python ots_manager.py create-user -u "guest_op" -p "TempPass123!" \\
  -g Operations:IN --app android --exp 30 --max 1

# Administrator account
python ots_manager.py create-user -u "tac_commander" -p "AdminPass!" \\
  -g Command:BOTH --admin

# Using the special group name `ALL`
When you want to associate a user with every group present on the
OpenTAKServer, use `ALL` as the group value. The utility expands `ALL`
into the full list returned by the server and performs the requested
associations. You can also combine it with a direction, such as `ALL:IN`
or `ALL:OUT`.

```bash
# Create a user and associate with every group on the server (default
direction: BOTH)
python ots_manager.py create-user -u "global_user" -p "Pass123!" -g ALL --
app android

# Associate an existing user with every group in the OUT direction
python ots_manager.py link -u "global_user" -g ALL --direction OUT

# Associate an existing user with every group in the IN direction
python ots_manager.py link -u "global_user" -g ALL --direction IN

# You can also use the syntax with direction in the group token itself
python ots_manager.py create-user -u "global_user_2" -p "Pass123!" -g
ALL:OUT --app android

# Run a batch where every record uses ALL or ALL:OUT/ALL:IN (see JSON
example below)
python ots_manager.py batch -f usuarios_all.json -o resultado_all.json
```

```

### 6.4 Direction when linking an existing user

```bash
# IN, OUT, or BOTH; the default is BOTH
python ots_manager.py link -u "pilot1" -g "Quick_Response_Force" -d IN
```

7. CLI commands

7.1 Help

```
python ots_manager.py --help
python ots_manager.py create-user --help
python ots_manager.py qr --help

### 7.2 List groups

```bash
python ots_manager.py list-groups
```

The command lists the groups returned by the OpenTAKServer. You can use the special group name `ALL` with `-g` (for example `-g ALL`) when calling `create-user`, `link`, or when a batch record includes `"groups": ["ALL"]`; the utility will expand `ALL` into the full set of groups and perform the requested associations for each.

```
### List users (admin and last_login)

```bash
python ots_manager.py list-users
```

The `list-users` command queries the server's user endpoints and attempts to display a concise summary for each user including:

- `username`
- whether the user is an administrator (detected from direct flags or the `roles` list)
- `last_login` (ISO or server-provided timestamp, `N/A` if not available)

The command handles common server response shapes and inspects the `roles` objects (name/permissions) to determine administrator privileges.

7.2 List groups

```
python ots_manager.py list-groups
```

The command lists the groups returned by OpenTAKServer. The special group value `ALL` can be used with `create-user`, `link`, or batch records. The utility expands it to the complete current group list before creating associations.

7.3 List users

```
python ots_manager.py list-users
```

The command displays a concise summary of each user, including `username`, administrator status, and `last_login` when available. Administrator status is detected from direct flags or the server's `roles` data.

7.4 The special group value `ALL`

Use `ALL`, `ALL:IN`, or `ALL:OUT` to associate a user with every group returned by the server:

```
python ots_manager.py create-user -u "global_user" -p "Pass123!" -g ALL --
app android
python ots_manager.py link -u "global_user" -g ALL --direction OUT
python ots_manager.py create-user -u "global_user_2" -p "Pass123!" -g
ALL:OUT --app android
```

For batch mode:

```
[
  {"username": "global_one", "password": "GPass1!", "groups": ["ALL:OUT"],
"app": "android"},
  {"username": "global_two", "password": "GPass2!", "groups": ["ALL:IN"],
"app": "iphone", "expiration": 14},
  {"username": "global_three", "password": "GPass3!", "groups": ["ALL"],
"app": "android"}
]
```

`ALL` without a suffix uses `BOTH`.

7.5 Create user e gerar QR Code Android

```
python ots_manager.py create-user \
  -u "piloto1" \
  -p "Senha123!" \
  -g CSAR \
  --app android
```

The command creates the group, creates the user, associates the group in the `IN` and `OUT`, requests the string from OTS, and saves a PNG with a default name such as `piloto1_android.png`.

7.3 Create user e gerar QR Code iPhone

```
python ots_manager.py create-user \
  -u "piloto2" \
  -p "Senha123!" \
  -g CSAR \
  --app iphone
```

7.4 Provide email and administrator profile

```
python ots_manager.py create-user \
  -u "operador" \
  -p "SenhaForte!" \
  -e "operador@empresa.com" \
  --admin \
  --app android
```

Use `--admin` somente quando a função operacional realmente exigir privilégios administrativos.

7.5 Validity and activation limit

```
python ots_manager.py create-user \
  -u "convidado" \
  -p "Senha123!" \
  -g Visitantes \
  --app android \
  --exp 30 \
  --max 1
```

`--exp 30` represents 30 days from the time of generation. A date in the format can also be used: `YYYY-MM-DD`:

```
python ots_manager.py qr -u "piloto1" --app android --exp 2026-12-31 --max
2
```

When `--exp` e `--max` are omitted, these fields are not sent and the server applies its default policies.

7.6 Generate a QR Code for an existing user

```
python ots_manager.py qr -u "piloto1" --app android
python ots_manager.py qr -u "piloto2" --app iphone --save-qr
piloto2_ios.png
```

7.7 Create group isolado

```
python ots_manager.py create-group -n "Patrulha"
```

7.8 Link a user to a group

```
python ots_manager.py link -u "piloto1" -g "Patrulha"
```

Specific direction:

```
python ots_manager.py link -u "piloto1" -g "Patrulha" --direction IN
python ots_manager.py link -u "piloto1" -g "Patrulha" --direction OUT
```

7.9 List groups and users

```
python ots_manager.py list-groups
python ots_manager.py list-users
```

`list-groups` queries the existing groups. `list-users` displays `username`, administrator status, and `last_login` when available.

7.10 Associate a user with every group

Use `ALL`, `ALL:IN`, or `ALL:OUT` as the `--group` or `--groups` value:

```
python ots_manager.py create-user -u "global_user" -p "Pass123!" -g ALL --
app android
python ots_manager.py create-user -u "global_user_out" -p "Pass123!" -g
ALL:OUT --app android
python ots_manager.py create-user -u "global_user_in" -p "Pass123!" -g
ALL:IN --app iphone
python ots_manager.py link -u "global_user" -g ALL --direction OUT
```

`ALL` without a suffix uses `BOTH`; `ALL:IN` uses only `IN`; `ALL:OUT` uses only `OUT`.

8. Batch provisioning

8.1 File `usuarios.json`

8.1.1 Groups with individual directions

Each `groups` item may be a plain string or an object with `name` and `direction`:

```
[
  {
    "username": "operator_alpha",
    "password": "SecurePassword123!",
    "groups": [
      {"name": "Group_IN", "direction": "IN"},
      {"name": "Group_OUT", "direction": "OUT"},
      {"name": "Bidirectional_Group", "direction": "BOTH"}
    ],
    "app": "android"
  }
]
```

`parse_group_entry()` accepts `IN`, `OUT`, and `BOTH`; a string such as `"CSAR"` is equivalent to an object with direction `BOTH`.

```
[
  {
    "username": "operator_alpha",
    "password": "SecurePassword123!",
    "email": "alpha@example.com",
    "administrator": false,
    "groups": [
      {"name": "CSAR", "direction": "BOTH"},
      {"name": "Rescue", "direction": "IN"},
      {"name": "Command", "direction": "OUT"}
    ],
    "app": "android"
  },
  {
    "username": "operator_bravo",
    "password": "SecurePassword456!",
    "email": "bravo@example.com",
    "administrator": false,
    "groups": [
      "CSAR",
      {"name": "Support", "direction": "IN"}
    ],
    "app": "iphone"
  },
  {
    "username": "temporary_guest",
    "password": "TempPassword789!",
    "administrator": false,
    "groups": [
      {"name": "Operations", "direction": "IN"}
    ],
    "app": "android",
    "expiration": 30,
    "max_uses": 1
  }
]
```

As chaves `expiration` and `max_uses` are optional. Se estiverem ausentes ou com valor `null`, as restrições não serão enviadas ao servidor.

8.2 Execution

```
python ots_manager.py batch \
  -f usuarios.json \
  -o resultado_qr_codes.json
```

Processing:

1. creates all groups found;
2. processes each user;
3. associates groups in `IN` and `OUT`;

4. converts validity to Unix Epoch when necessary;
5. requests the QR Code;
6. saves PNGs in `qrcodes/`;
7. exports the consolidated report.

Batch example: using `ALL` with a direction

Example input file (`usuarios_all.json`):

```
[
  {
    "username": "global_one",
    "password": "GPass1!",
    "groups": ["ALL:OUT"],
    "app": "android"
  },
  {
    "username": "global_two",
    "password": "GPass2!",
    "groups": ["ALL:IN"],
    "app": "iphone",
    "expiration": 14
  },
  {
    "username": "global_three",
    "password": "GPass3!",
    "groups": ["ALL"],
    "app": "android"
  }
]
```

When processed, each `ALL:OUT` or `ALL:IN` entry is expanded to the current group list from `list_groups()`, and each group is linked in the specified direction. `ALL` without a suffix uses the default direction `BOTH`.

8.4 Batch with every group and a direction

```
[
  {"username": "global_one", "password": "GlobalPass1!", "groups":
["ALL:OUT"], "app": "android"},
  {"username": "global_two", "password": "GlobalPass2!", "groups":
["ALL:IN"], "app": "iphone", "expiration": 14},
  {"username": "global_three", "password": "GlobalPass3!", "groups":
["ALL"], "app": "android"}
]
```

```
python ots_manager.py batch -f usuarios_all.json -o resultado_all.json
```

`ALL:OUT` uses only `OUT`; `ALL:IN` uses only `IN`; `ALL` without a suffix uses `BOTH`.

8.3 Report structure

```
[
  {
    "username": "operador_alpha",
    "app": "android",
    "max_uses": "server default",
    "expiration": "server default",
    "qr_string": "string returned by OTS",
    "qr_image": "qrcodes/operador_alpha_android.png"
  }
]
```

The report must be treated as sensitive material. In real environments, consider generating an operational version without the `qr_string` key for sharing.

9. QR Codes and access validity

9.1 Android

The Android endpoint receives `username`. The optional fields are:

Field	Meaning
`exp`	expiration instant in Unix Epoch
`nbf`	instant from which the QR is valid
`max`	activation limit

When existe expiração, o script calcula `nbf` as the current instant in UTC.

9.2 iPhone

The iPhone endpoint is queried using `GET` and directly returns the iTAK configuration string. The response may be JSON, with keys such as `qr_string` ou `itak_qr_string`, or plain text.

9.3 Date conversion

The `parse_expiration` function accepts:

- number of days, such as `30`;
- absolute date, such as `2026-12-31`;
- empty values or values equivalent to `none`, `null`, `eternal` e `infinite`, treated as no restriction.

Absolute dates are interpreted in UTC. Confirm the timezone and server policy before using expiration dates in critical operations.

10. Operational procedures

10.1 Provision a new team

1. confirm the OTS address and operating account;
2. list the required groups;
3. review the JSON with another operator;
4. run the batch during an authorized window;
5. check the report and PNGs;
6. distribute each QR Code only to the correct recipient;
7. test login and CoT traffic on the device;
8. record the execution and validity date.

10.2 Reissue a QR Code

```
python ots_manager.py qr \
  --username "usuario_existente" \
  --app android \
  --save-qr "reemitido_usuario_android.png"
```

Confirm whether reissuing invalidates the previous QR Code according to OTS policy.

10.3 Validate post-provisioning connectivity

On the device:

- import the QR Code into the corresponding application;
- confirm the server address;
- confirm that the user can authenticate;
- test receipt of a CoT message;
- test sending a CoT message;
- validate that the expected groups are applied.

11. Error handling and diagnostics

Connection failure

Symptom: erro de conexão ou timeout.

Checks:

```
curl -i "$OTS_URL/"
getent hosts <hostname>
```

Confirm the IP, port, firewall, VPN, and that the OTS service is active.

Authentication failure

Symptom: mensagem `Falha na autenticação` com status HTTP diferente de `200` ou `201`.

Check the username, password, URL, login method, and whether the account is enabled.

HTTP 400 ao criar grupo ou usuário

This may indicate invalid data or duplication. O script reconhece duplicidade quando o corpo contém `exists`; otherwise, read the complete response and correct the submitted fields.

HTTP 401 ou 403 in a protected operation

Check:

- whether login actually created the session;
- whether the CSRF token exists in the cookies or response;
- whether the headers `X-CSRFToken` e `X-CSRF-TOKEN` were sent;
- se `Referer` e `Origin` match the `OTS_URL`;
- whether the account has permission for the operation;
- whether the session has expired.

Empty or invalid QR Code

Confirm the selected endpoint (`android` ou `iphone`), the user, HTTP status, and response format. Preserve the returned body during diagnostics without exposing tokens in public logs.

PNG not generated

Confirm the installation of `qrcode` and `pillow`, the existence of the destination directory and write permissions:

```
python -c "import qrcode, PIL; print('dependências OK')"
```

Partially completed batch

The batch may create some resources before failing on a later record. Do not rerun blindly. Compare the report, check which users and groups exist, and handle duplicates in a controlled manner.

12. Best practices and limitations

- Create a secure backup do arquivo de entrada antes de uma execução em lote.
- Use stable usernames and documented conventions.
- Avoid spaces and ambiguous characters in identifiers.
- Test first with a laboratory account.
- Do not distribute QR Codes through public channels.
- Record the generation date, validity, and person responsible for delivery.
- Always validate both group directions when there is bidirectional traffic.
- The `BOTH` mode executes two requests; one failure may leave the association incomplete.
- Handling “already exists” does not replace checking the current state on the server.
- The endpoint and response format may vary depending on the OpenTAKServer version.
- The original script does not implement transactional rollback. For critical batches, use smaller stages and subsequent reconciliation.

- The password appears on the command line in the examples; in production, prefer a secure input mechanism.

12.1 Notes de manutenção do código

When transcribing or updating the source code, validate the indentation of the block `get_qr_string`, especially the branches `elif app_type == "iphone"` e `else`. Run a syntax compilation before use:

```
python -m py_compile ots_manager.py
```

It is also recommended to add tests for `parse_expiration`, seleção dos endpoints, tratamento de respostas JSON/texto e montagem dos payloads.

13. Operations checklist

Before execution

- [ ] `OTS_URL` aponta para o ambiente correto.
- [ ] O acesso de rede foi validado.
- [ ] A conta tem permissão suficiente.
- [ ] A senha não está versionada.
- [ ] O JSON foi validado e revisado.
- [ ] Os nomes de grupos e usuários estão corretos.
- [ ] `app` é `android` ou `iphone`.
- [ ] Expiração e limite de uso foram conferidos.

After execution

- [ ] Login was successful.
- [ ] Groups were created or confirmed.
- [ ] Users were created or confirmed.
- [ ] Each group was associated in `IN` and `OUT`.
- [ ] The QR string was returned.
- [ ] The PNG was generated and opened for review.
- [ ] The JSON report was saved in a protected location.
- [ ] The device was tested in both communication directions.

14. Quick reference

```
# Instalação
pip install requests qrcode pillow

# Configuração
export OTS_URL="http://servidor-ots"
export OTS_USER="admin"
export OTS_PASS="..."

# Usuário Android
python ots_manager.py create-user -u piloto1 -p 'Senha123!' -g CSAR --app
android

# Usuário iPhone
python ots_manager.py create-user -u piloto2 -p 'Senha123!' -g CSAR --app
iphone

# QR existente
python ots_manager.py qr -u piloto1 --app android --save-qr piloto1.png

# Grupo
python ots_manager.py create-group -n Patrulha

# Associação bidirecional
python ots_manager.py link -u piloto1 -g Patrulha

# Lote
python ots_manager.py batch -f usuarios.json -o resultado_qr_codes.json

# Validação sintática
python -m py_compile ots_manager.py

# List groups and users
python ots_manager.py list-groups
python ots_manager.py list-users

# Create a user in every group with a specific direction
python ots_manager.py create-user -u global_user -p 'Pass123!' -g ALL:OUT
--app android
python ots_manager.py create-user -u global_user_in -p 'Pass123!' -g
ALL:IN --app iphone

# Associate an existing user with every group
python ots_manager.py link -u global_user -g ALL --direction BOTH
```

15. Appendix: code structure

The `ots_manager.py` file is organized around the following functions:

Function	Responsibility
`get_csrf_token()`	Retrieves the CSRF token from the session
`login()`	Authenticates and updates headers
`create_group()`	Creates or confirms a group
`create_user()`	Creates or confirms a user
`add_user_to_group()`	Associates groups in `IN`, `OUT` ou `BOTH`
`parse_expiration()`	Converts days/date to Unix Epoch
`get_qr_string()`	Gets the Android or iPhone configuration
`save_qr_code_image()`	Generates and saves the PNG image
`list_groups()`	Retrieves the current server group list
`list_users()`	Retrieves users and summarizes admin/last-login data
`list_groups()`	Retrieves the current server groups
`list_users()`	Lists users, administrator status, and last login
`process_batch_list()`	Orchestrates batch provisioning and expands `ALL`
`main()`	Defines the CLI and dispatches commands

Conclusion

OTS Manager reduces manual tasks and standardizes OpenTAKServer provisioning. Automation must be used together with access control, credential protection, input-file review, and functional validation on the ATAK/iTAK device.

OTS Manager Operations Manual — Version 1.0.1
Created by Orlando Nascimento Santos — onascimento@gmail.com

> **Note:** the complete `ots_manager.py` source is maintained separately in the repository. This manual documents its operation, commands, payloads, and results.